Autoencoder y clasificador convolucional profundo sobre Fashion-MNIST y con pre-entrenamiento

Ignacio Benjamín Ceballos*
Facultad de Matemática, Astronomía, Física y Computación,
Universidad Nacional de Córdoba, Ciudad Universitaria, 5000 Córdoba, Argentina

Los autoencoders son un tipo de red neuronal que comprime datos en representaciones compactas(codificación) y los reconstruyen (decodificación), minimizando una función de error, para lograr aprendizaje no supervisado. En este trabajo los utilizamos junto con capas convolucionales, que aplican filtros locales (kernels), pooling y activaciones, para extraer patrones relevantes en imágenes. Este trabajo implementa dos autoencoders convolucionales sobre el conjunto de datos Fashion-MNIST. Uno de ellos con cuello de botella(bottleneck) para reducción dimensional y otro sin bottleneck con capas convolucionales mas grandes. El codificador extraído de los autoencoders desarrollados se integra a una red neuronal MLP clasificadora y se comparan cuatro esquemas de entrenamiento que combinan o descartan el pre-entrenamiento del codificador. Con los resultados se analiza el desempeño de cada una de estas redes teniendo en cuenta eficiencia computacional y precisión.

I. Introducción

Las redes neuronales convolucionales (CNN) han demostrado gran desempeño en tareas de clasificación de imágenes por su capacidad para explotar la estructura espacial de los datos.[1] A su vez, los autoencoders se utilizan para aprender representaciones compactas de los datos de manera no supervisada[2]. Estos comprimen la información y luego intentan reconstruirla minimizando una función de error, en este caso el error cuadrático medio (MSE). Sabiendo esto, y dada la naturaleza del problema, es muy conveniente hacer uso de redes convolucionales para armar la estructura del autoencoder que queremos utilizar.

En este trabajo se incorpora la parte codificadora de autoencoders convolucionales en una red neuronal clasificadora feed forward multicapa para evaluar el impacto que tiene su uso al clasificar imagenes de Fashion-MNIST.[3] Se utilizara uno con bottleneck para lograr una representación más comprimida y otro sin bottleneck con mayor expresividad.

Los cuatro esquemas de entrenamiento que se evaluaron son:

0. Entrenamiento del clasificador con codificador pre-entrenado: se entrena solo el clasificador, el codificador que ya fue pre-entrenado no se continua entrenando en esta etapa.

1. Entrenamiento completo sin pre-entrenamiento: se entrena la red clasificadora completa partiendo de una red completamente sin entrenar, es decir, con codificador y clasificador inicializados aleatoriamente.

2. Entrenamiento completo con codificador pre-entrenado: se entrena toda la red clasificadora (codificador + clasificador) partiendo de un codificador pre-entrenado en el autoencoder y un clasificador sin entrenar.

3. Entrenamiento solo del clasificador sin pre-entrenamiento del codificador: se entrena únicamente el módulo clasificador utilizando un codificador sin entrenar (inicializado aleatoriamente).

II. Teoría

El término convolución hace referencia a la combinación matemática de dos funciones para producir una tercera. Fusiona dos conjuntos de información. En el caso de las CNN, estas aplican un filtro, también llamado kernel que se combina con la entrada para obtener un feature map(mapa de características).

Mas detalladamente, el filtro se desplaza sobre la imagen y se va aplicando una multiplicación matricial en cada desplazamiento, cada uno de estos resultados se guardan en un slot del feature map .

Se aplican numerosas convoluciones a la entrada, y cada operación usa un filtro diferente, para extraer diferentes características. Al final se podrán juntar todos los feature maps y hacerlos salida de la capa convolucional. Como cualquier otra red neuronal, se usan funciones de activación para hacer que la salida sea no lineal. En el caso de las CNN la salida de la convolución pasará por esta función de activación, en este trabajo se utiliza ReLu.[4][5]

Parámetros que se ajustaron en la red convolucional:[6]

- Kernel: se ajusta su tamaño.
- Stride: El desplazamiento que hace el filtro de convolución sobre la imagen.
- Padding: valores agregados alrededor de la entrada para controlar el tamaño del mapa de salida.
- Cantidad de filtros: determina la cantidad de canales de salida.

Entre capas convolucionales se suele agregar una capa de pooling, cuya función principal es reducir continua-

mente el tamaño de parámetros y computación en la red. En este trabajo se utiliza max pooling, que toma el valor máximo en cada ventana, el tamaño de esta ventana es ajustable y también lo es el tamaño del desplazamiento(stride) de dicha ventana.[7]

En la decodificación se suelen emplear capas de convolución traspuesta. Estas capas permiten aumentar la dimensión espacial del mapa de características, expandiendo progresivamente su tamaño. Si bien se las denomina "traspuestas", no revierten exactamente la operación de convolución, sino que realizan una operación de expansión controlada que facilita la reconstrucción de la imagen original.[8]

A. Valores de los hiperparámetros del sistema y arquitectura de los autoencoders utilizados

En todos los experimentos se mantuvo fijo un conjunto de hiperparámetros, elegidos principalmente a partir de resultados empíricos y criterios prácticos de eficiencia. El tamaño de batch se fijó en 256, ya que para el hardware utilizado ofreció el mejor compromiso entre velocidad de entrenamiento y estabilidad del gradiente. El autoencoder se entrenó con una tasa de aprendizaje de 10^{-3} para asegurar una convergencia estable durante la optimización de la reconstrucción. El clasificador se entrenó con una tasa de aprendizaje de 10^{-2}, ya que los gradientes de la entropía cruzada suelen ser estables y permiten actualizaciones relativamente grandes durante el entrenamiento.

En la red clasificadora se aplicó un dropout de 0.3, valor que produjo mejor generalización en las primeras pruebas, mientras que en los autoencoders se utilizó un dropout de 0.2. Respecto a los optimizadores, la red clasificadora se entrenó con SGD, ya que ofreció mejor desempeño final, mientras que los autoencoders se entrenaron con Adam para favorecer una convergencia más rápida y estable en la tarea de reconstrucción. Esos fueron los optimizadores que dieron los mejores resultados en cada caso. Con respecto a la cantidad de épocas de entrenamiento, se usan 50 épocas para cada autoencoder y 100 épocas para la red clasificadora. Esto se debe a que generalmente se notó que la curva era relevante hasta esas épocas ya que todavía había cierta convergencia en cada caso.

Estructura del autoencoder 1:

```
Encoder:
Conv2d(in=1,out=16,ker=3x3,pad=1) -> ReLU ->
MaxPool(ker=2x2,stride=2) ->
Conv2d(in=16,out=32,ker=3x3, pad=1) -> ReLU ->
MaxPool(ker=2x2,stride=2) ->
Flatten(32*7*7) -> Linear(in=1568,out=128)

Decoder:
Linear(in=128,out=1568) -> Unflatten(32x7x7)
ConvTranspose2d(in=32,out=16,ker=2x2,stride=2) ->
ReLU ->
ConvTranspose2d(in=16,out=1,ker=2x2,stride=2) ->
Sigmoid
```

El bottleneck de 128 dimensiones fuerza compresión extrema (de 784 píxeles originales a 128 features). Al experimentar con diversas estructuras mas profundas o con capas mas grandes con cuello de botella, se observó que estructuras grandes no ayudan a que la pérdida continúe bajando,y su tiempo de entrenamiento es peor. Una de las razones es que pasa por una etapa de gran compresión en la que inevitablemente se pierde información; ya que hay una reducción de un 84 % con respecto a la dimensión de la entrada, y por lo tanto hay un punto en el que no podemos retener mas características, y grandes capas convolucionales con mayor profundidad dejan de tener utilidad si no se aumenta este cuello de botella para que pasen mas características. La otra razón que puede explicar esto es el fenómeno de supresión del gradiente[5].

Estructura del autoencoder 2:

```
Encoder:
Conv2d(in=1,out=32,ker=3x3, pad=1) -> ReLU ->
MaxPool(ker=2x2,stride=2) -> Dropout(0.2)
Conv2d(in=32,out=64,ker=3x3,pad=1) -> ReLU ->
MaxPool(ker=2x2,stride=2)

Decoder:
ConvT2d(in=64,out=32,ker=2x2, stride=2) -> ReLU
ConvT2d(in=32,out=1,ker=2x2, stride=2) -> Sigmoid
```

De 784 píxeles originales extrae 3136 características intermedias en un tensor de 64 canales de 7x7 (4x expansión), permitiendo mayor poder expresivo contra el autoencoder 1. Sin embargo, la experimentación reveló que capas convolucionales adicionales deterioran convergencia, resultando en pérdida estancada y reconstrucciones peores, con mayores tiempos de entrenamiento. Este comportamiento podría estar relacionado con dificultades en la optimización del modelo, como el fenómeno de supresión del gradiente.

Estructura del clasificador:

```
Entrada: encoder(x) -> [*]
Linear(in=vector_size,out=n1) -> ReLU
-> Dropout(0.2)

Linear(in=n1,out=n2) -> ReLU
-> Dropout(0.2)

Linear(in=n2,out=10)  [10 clases Fashion-MNIST]

[*] si es encoder2 primero se debe usar Flatten
para que sea un vector
```

Se usaron n1 = 256 y n2 = 64; vector_size es el tamaño de la salida aplanada del codificador. Para Encoder1 usa Linear(128,n1) directamente. Para Encoder2 se agrega

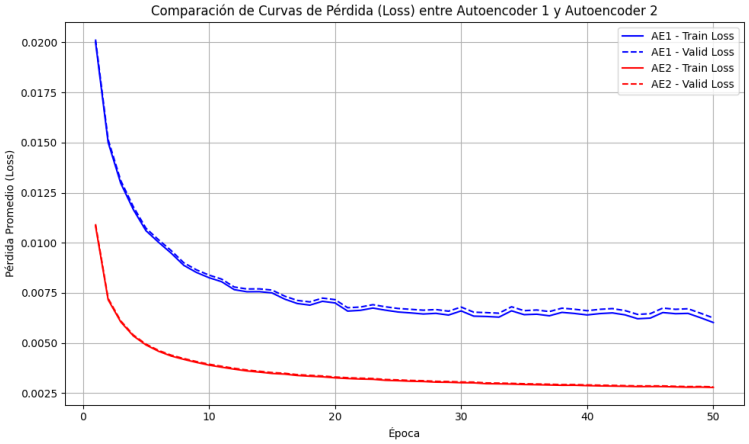
Flatten antes de Linear(3136,n1) para convertir el tensor a un vector. Las capas denominadas como Linear corresponden a capas densas o fully connected. Para su entrenamiento se utilizó CrossEntropyLoss + LogSoftMax.

III. Resultados

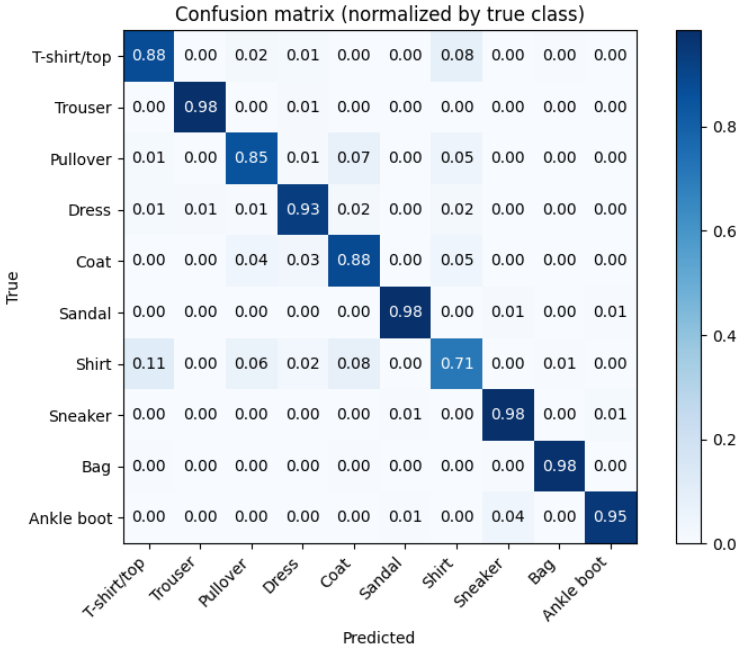
Aclaraciones: Las curvas "Incorrectas" representan métricas calculadas por batch durante el entrenamiento. Dado que no reflejan el rendimiento global del modelo, no se utilizan para el análisis de resultados ni para la comparación con la validación. Se considerara buen desempeño cuando exhibe valores bajos de la función de pérdida y altos niveles de precisión sobre el conjunto de validación, eso refleja adecuada capacidad de generalización para la clasificación de las prendas. Se incluyeron los gráficos correspondientes al esquema más efectivo (2) y al que presenta el comportamiento más atípico (3). No obstante, se analiza a continuación el comportamiento observado en cada uno de los esquemas evaluados.

Tabla I: Resultados promedio de entrenamiento y validación para los distintos experimentos al final del entrenamiento

enc	Esq.	Train Loss	Val Loss	Train Prec.	Val Prec.
1	0	0.2842	0.3266	89.4 %	87.9 %
1	1	0.3456	0.3798	87.1 %	85.9 %
1	2	0.2330	0.2802	91.2 %	89.6 %
1	3	0.9716	0.9790	63.2 %	62.5 %
2	0	0.2026	0.2618	92.6 %	90.4 %
2	1	0.2943	0.3426	89 %	87.2 %
2	2	0.1639	0.2365	94 %	91.3 %
2	3	0.4823	0.5036	82.8 %	82 %

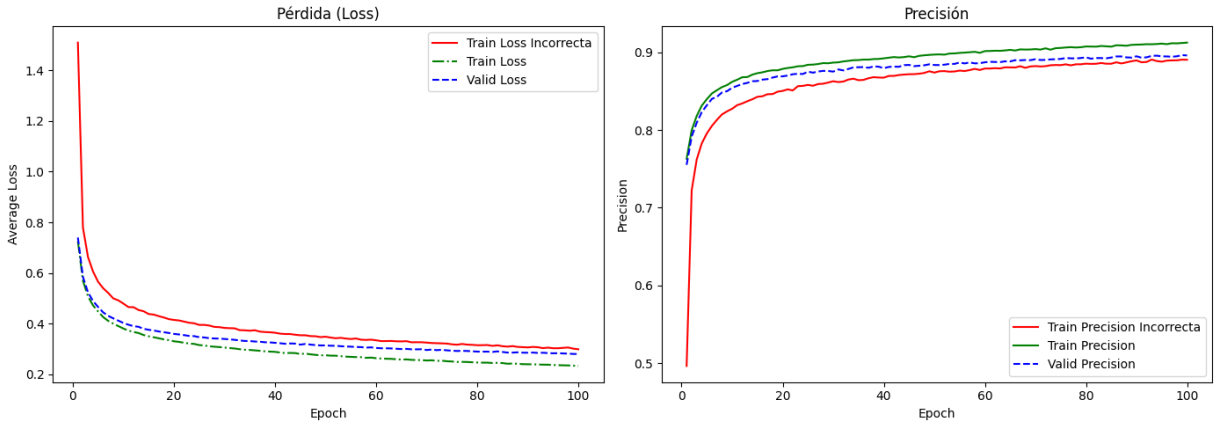


AE1 vs AE2

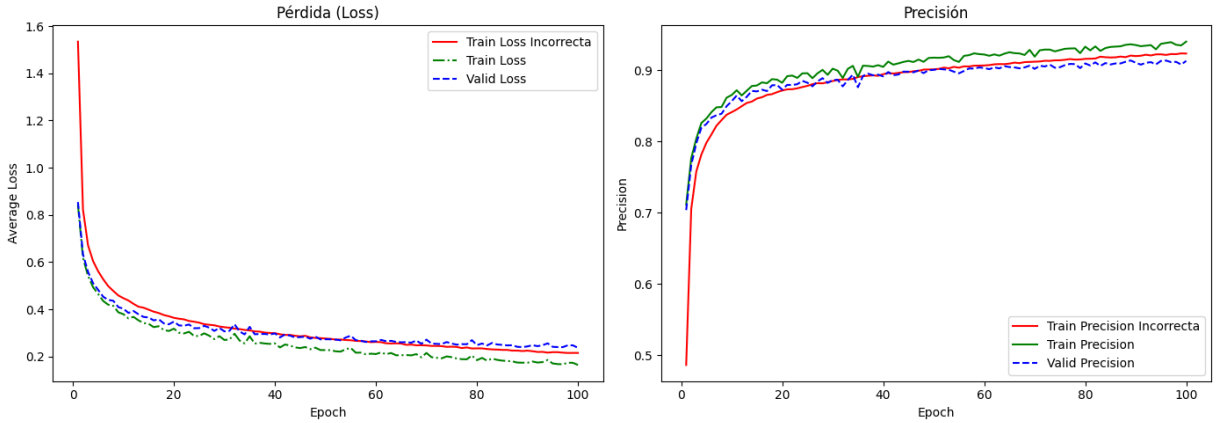


Matriz de confusión AE2: Esquema 2

Esquema de entrenamiento 2

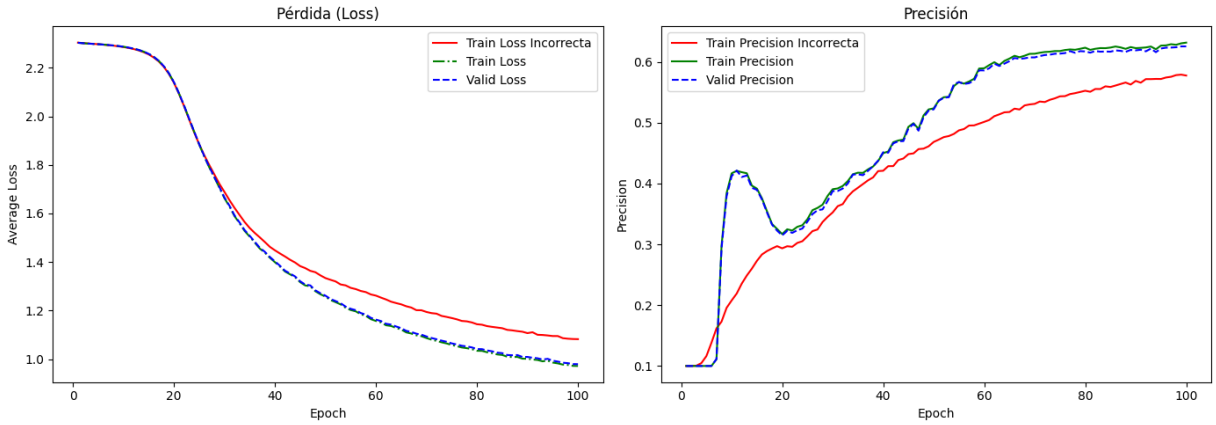


(a) Modelo con encoder1

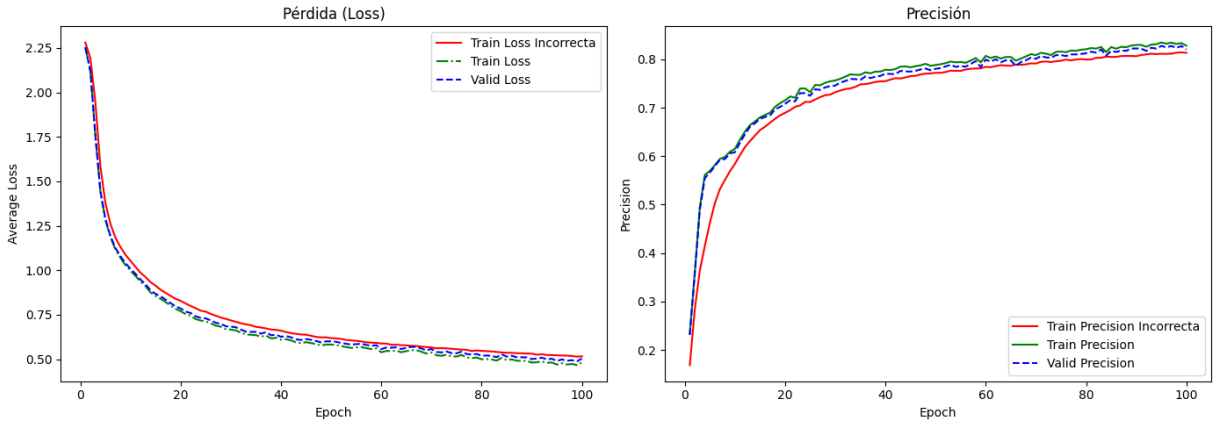


(b) Modelo con encoder2

Esquema de entrenamiento 3



(c) Modelo con encoder1



(d) Modelo con encoder2

IV. Discusión

En todos los esquemas el mejor valor de precisión en la validación coincidió con el valor que se alcanzó al finalizar el proceso de entrenamiento.

El ligero ruido que presentan las curvas de pérdida y precisión del modelo con el encoder 2 en todos los gráficos, puede deberse a la combinación de entrenamiento estocástico y al hecho de que el modelo posee más parámetros, lo que permite ajustes más finos y, en consecuencia, oscilaciones más visibles durante el aprendizaje.

- **Esquema 0:** El esquema 0 presenta un comportamiento similar al esquema 2 en cuanto a la forma de las curvas, aunque con una velocidad de convergencia menor. En el modelo con encoder 1, a partir de la época 80 la mejora en el error de validación se vuelve marginal, observándose un estancamiento de unas cinco épocas y un incremento de precisión de solo 87.6 % a 87.9 % en las 20 épocas siguientes. En contraste, el modelo con encoder 2 converge más rápidamente y no muestra un estancamiento pronunciado: alcanza una precisión de validación del 88.8 % en la época 90, que aumenta hasta 90.4 % diez épocas después. Esto sugiere que el modelo aún podría beneficiarse de más épocas de entrenamiento. Tras 100 épocas, se obtiene una precisión comparable a la del clasificador original sin encoder.

- **Esquema 1:** El comportamiento al usar este esquema en ambos modelos se caracteriza por una convergencia lenta, lo cual es esperable dado que debe entrenar desde 0 la red completa, se puede ver mayor ruido en ambos casos, nuevamente indicando que la cantidad de parámetros que deben ser entrenados influye en la aparición de estas oscilaciones. Con el encoder 2 tenemos el único caso donde se presenta overfitting claro, a partir de la época 70 hay un estancamiento, 30 épocas mas no significaron ningún cambio, dando exactamente la misma precisión. Por esta razón el entrenamiento en este caso tiene solo sentido hasta la época 70.

- **Esquema 2:** Se puede observar que el esquema de entrenamiento 2 es el que mejor resultados ha dado, como se puede ver en la figura (a), con ambos autoencoders se observa una convergencia rápida durante las primeras épocas, seguida de una estabilización progresiva de la pérdida y la precisión, lo que indica un proceso de entrenamiento estable bajo este esquema. Ninguno de los modelos presenta sobre ajuste evidente. Se puede ver una ligera separación de la curva de validación y la de entrenamiento en ambos casos pero que no llega a verse reflejado en un estancamiento notable para la época 100. Bajo este esquema el modelo con encoder 2 consiguió una precisión mejor que el clasificador original (que no utiliza encoders).

- **Esquema 3:** Con el codificador 1 se aprecia en la curva la dificultad en las primeras 20 épocas de tener que identificar patrones aleatorios en una entrada de solo 128 valores; es sorprendente que aun así el modelo logra converger. Al extender el entrenamiento a 150 épocas se logra alrededor de 70 % de precisión. El modelo tarda en converger pero logra una predicción competente para información que parece ser aleatoria y demasiado reducida. El pico pronunciado en la precisión de validación puede interpretarse como una fluctuación transitoria del proceso de optimización, donde el modelo alcanza temporalmente una solución favorable que luego es descartada al continuar el entrenamiento, dando lugar a una convergencia más estable aunque más lenta. El comportamiento del modelo con encoder 2 muestra muy buenos resultados dado el contexto y comparando con el encoder 1. Evidentemente es mas sencillo para el clasificador identificar patrones sobre una entrada con mayor información. Este último logró clasificar las imágenes con una efectividad muy buena, incluso mejor que algunos modelos de clasificador sin encoder.

V. Conclusión

En general, los resultados muestran un mejor desempeño al utilizar el encoder 2, lo cual es esperable dada su mayor capacidad representacional, pero la mejora se obtiene a costa de un mayor costo computacional, asociado al incremento en la dimensionalidad del espacio de características aprendido. Por el contrario, el encoder 1, aunque presenta un desempeño inferior, alcanza resultados competitivos utilizando una representación considerablemente más compacta. En particular, el hecho de que el clasificador logre una precisión del 89.6 % en el experimento 2 empleando únicamente el 16 % de la información de la imagen original resulta notable, y sugiere que una fracción reducida de las características es suficiente para capturar información relevante para la tarea. Asimismo, se observa que el uso del encoder 2 permite mejorar la precisión respecto de un clasificador entrenado sin codificador. En términos generales, la incorporación de codificadores en la arquitectura proporciona un mayor control sobre la información de entrada, facilitando la extracción de características relevantes y permitiendo ajustar el compromiso entre precisión y eficiencia según los requerimientos del problema. Finalmente, los resultados indican que el pre-entrenamiento del codificador facilita y acelera el proceso de entrenamiento. No obstante, también se evidencia que es posible alcanzar desempeños satisfactorios incluso sin pre-entrenamiento o entrenamiento alguno de los codificadores.

* benjamin.ceballos@mi.unc.edu.ar

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, *ImageNet Classification with Deep Convolutional Neural Networks* (Advances in Neural Information Processing Systems 25 (NIPS 2012), 2012).

[2] G. E. Hinton and R. R. Salakhutdinov, *Reducing the Dimensionality of Data with Neural Networks* (Science, 2006).

[3] H. Xiao, K. Rasul, and R. Vollgraf, Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms (2017), arXiv:1708.07747 [cs.LG].

[4] D. Cornelisse, An Intuitive Guide to Convolutional Neural Networks (2018), accedido el 13 de diciembre de 2025.

[5] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, Dive into deep learning (2023), arXiv:2106.11342 [cs.LG].

[6] PyTorch Developers, torch.nn.Conv2d (2025), documentación oficial de PyTorch. Accedido el 13 de diciembre de 2025.

[7] PyTorch Developers, torch.nn.MaxPool2d (2025), documentación oficial de PyTorch. Accedido el 13 de diciembre de 2025.

[8] PyTorch Developers, torch.nn.ConvTranspose2d (2025), documentación oficial de PyTorch. Accedido el 13 de diciembre de 2025.